

AI Incident Commander — Deep-Dive Technical Documentation

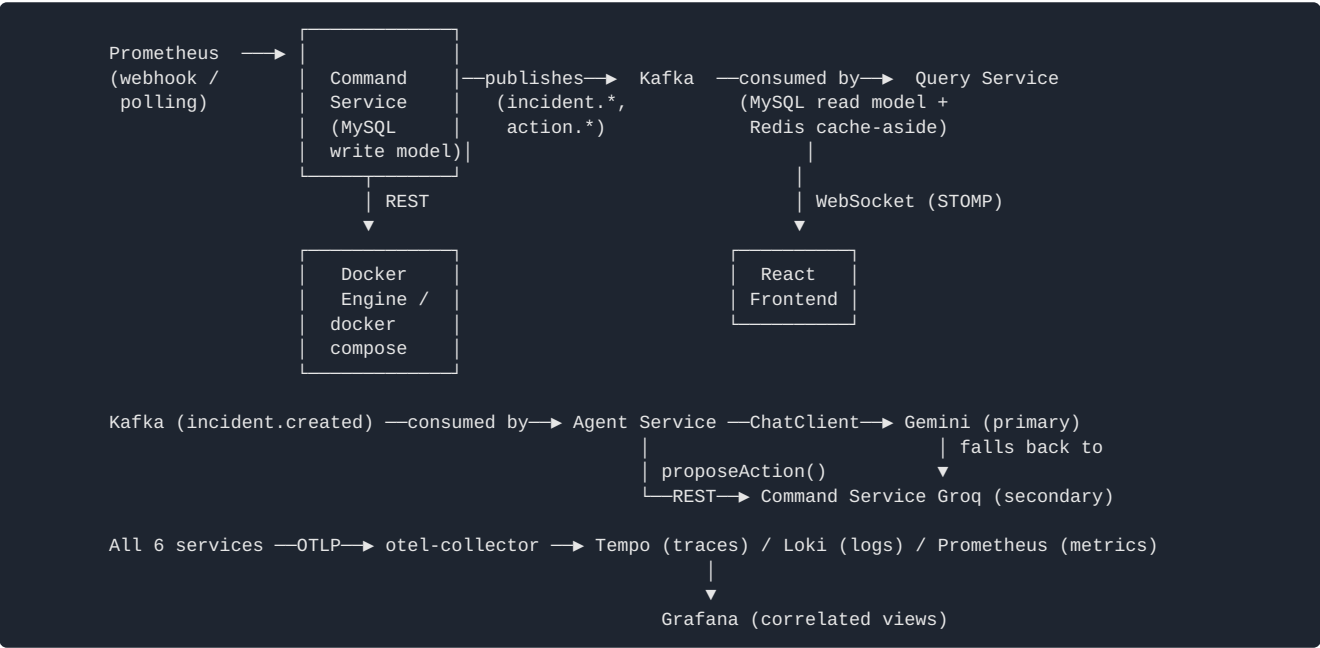
1. High-Level Architecture
2. Why This Shape? — The Core Design Patterns
3. Distributed Tracing — How It Actually Works
4. Resilience — The Full Picture
5. How Docker Execution Actually Works
6. End-to-End Walkthroughs
7. Design Trade-offs — Summary Table
8. Interview Question Bank
9. Quick-Reference: “What pattern is this?” Cheat Sheet

AI Incident Commander — Deep-Dive Technical Documentation

A self-healing, AI-assisted incident management platform. Prometheus fires an alert → an incident is opened → an AI agent investigates using read-only diagnostic tools → it proposes a remediation → a human approves → the platform executes a **real** side effect against Docker/Kafka → the system watches the outcome and can roll it back. Every hop is traced, logged, and metriced end-to-end.

This document explains **how it actually works under the hood** — the patterns used, why they were chosen, what breaks if you remove them, and the honest trade-offs. It ends with an interview-question bank mapped to each concept, so you can defend every design decision out loud.

1. High-Level Architecture



Services (all Spring Boot 4.1.0 / Java 21):

Service	Port	Responsibility	Datastore
api-gateway	18080	Single entry point: JWT auth, rate limiting, request routing	none (stateless)
command-service	18081	Write side: incident/action state machine, real Docker + Kafka execution	MySQL (write model)
query-service	18082	Read side: denormalized read models, Redis cache, WebSocket push, DLQ admin	MySQL (read model) + Redis
agent-service	18083	AI investigation, tool-calling, Prometheus anomaly polling	Redis (metric snapshot cache)
notification-service	18084	Email alerts on incident lifecycle events	MySQL (notification log)
frontend	5174	React dashboard, incident timeline, DLQ admin UI, live telemetry	—

Infra: MySQL, Redis, Kafka + Zookeeper, and a full observability stack (OTel Collector, Tempo, Loki, Prometheus, Grafana) — 15 containers total via one `docker-compose.yml`.

2. Why This Shape? — The Core Design Patterns

2.1 CQRS (Command Query Responsibility Segregation)

`command-service` and `query-service` are **physically separate services with separate databases**, not just separate classes in one app.

- **Command Service** owns the source of truth (MySQL write model: `Incident`, `RemediationAction`). It's the only service allowed to mutate state. Every mutation runs inside a `@Transactional` method, validates a state-machine transition, persists, and then publishes a Kafka event describing what happened.
- **Query Service** never touches the write database. It owns its *own* MySQL tables (`IncidentReadModel`, `ActionReadModel`) that are pure projections, rebuilt by replaying Kafka events. All reads — REST and WebSocket — are served from this side, backed by a Redis cache-aside layer.

Why split them at the service level instead of just splitting read/write repositories in one app? - Independent scaling: query traffic (dashboard polling/streaming) is usually far higher volume than command traffic (approve/execute clicks), so they can be scaled independently. - Fault isolation: if the AI/Docker-execution path in Command Service is under load or degraded, dashboards still render from Query Service's cache. - Schema freedom: the read model can be shaped exactly for UI queries (`getActiveIncidents`, `getByStatus`, `getByServiceName`) without dragging normalization/transaction concerns from the write side.

The cost: eventual consistency. A `POST /actions/{id}/approve` returns `200` from Command Service the instant the write DB transaction commits — but the dashboard, which reads from Query Service, won't reflect it until the Kafka event is consumed (typically single-digit milliseconds locally, but *not* the same instant). The frontend compensates for this gap with **optimistic UI updates + WebSocket push** rather than polling and hoping.

2.2 Event-Driven Architecture via Kafka

Every state transition in Command Service is published as a Kafka event (`incident.created`, `action.proposed`, `action.approved`, `action.executed`, `action.rejected`, `action.rolled_back`, `incident.escalated`, `incident.resolved`, `incident.status_updated`). Three independent consumer groups react to the same event stream for three different purposes:

Consumer group	Service	Purpose
<code>query-service-group</code>	Query Service	Update read model + evict cache + push WebSocket
<code>agent-service-group</code>	Agent Service	Trigger AI investigation on <code>incident.created</code>
(notification)	Notification Service	Send email alerts

This is the classic **pub/sub fan-out** — Command Service doesn't know or care who's listening. Adding a fourth consumer (e.g., a Slack-notifier service) requires zero changes to Command Service.

Partitioning key = `incidentId`. `kafkaTemplate.send(topic, String.valueOf(incidentId), event)` guarantees all events for one incident land in the same partition, and Kafka guarantees **in-order delivery within a partition**. This is what makes the sequence-number dedup logic (below) meaningful — without a consistent partition key, two events for the same incident could be consumed out of order by design.

2.3 Finite State Machine for Incident Lifecycle

`IncidentService` hardcodes a transition table:

```
NEW          → INVESTIGATING | RESOLVED | ESCALATED
INVESTIGATING → ACTION_PROPOSED | RESOLVED | ESCALATED
ACTION_PROPOSED → WAITING_APPROVAL | RESOLVED | ESCALATED
WAITING_APPROVAL → EXECUTING | RESOLVED | ESCALATED | INVESTIGATING
EXECUTING      → MONITORING | RESOLVED | ROLLBACK | ESCALATED
MONITORING     → RESOLVED | ROLLBACK | ESCALATED
ROLLBACK       → INVESTIGATING | RESOLVED | ESCALATED
RESOLVED, ESCALATED → (terminal)
```

Every service-layer method that changes status calls a single private `transitionStatus()` that checks this map and throws `InvalidStateTransitionException` on an illegal jump. This is deliberately **not** an enum with behavior (the GoF State pattern with polymorphic classes) — it's a data-driven table. That trade-off is worth defending in an interview:

- **Pro:** the entire lifecycle is visible in one place; adding a transition is a one-line change; trivially unit-testable (just assert the map).
- **Con:** transition *side effects* (what happens when you enter `EXECUTING`) still live scattered across each public method (`executeAction`, `rollbackAction`, etc.) rather than centralized in per-state `onEnter()` hooks — so behavior and legality are decoupled and you have to

read two places to fully understand one transition.

Action-level states (`ActionStatus` : `PROPOSED` → `APPROVED` → `EXECUTED` → `ROLLED_BACK` , or `PROPOSED` → `REJECTED`) are a second, smaller state machine nested inside the incident's.

2.4 Idempotency Keys (Redis-backed)

`POST /actions/{id}/approve` and `POST /actions/{id}/execute` both accept an optional `X-Idempotency-Key` header. The controller:

1. Looks up `idempotency:{key}` in Redis.
2. If present → deserialize and return the **cached response** immediately, without touching the service layer at all.
3. If absent → run the real operation, then store the serialized response under that key with a **24-hour TTL**.

This solves the classic “operator double-clicks Approve, or the frontend retries a request after a network timeout” problem. Without it, a retried `execute` call could restart a container twice, or scale replicas twice, because Docker/Kafka side effects are not naturally idempotent the way a SQL `UPDATE` is.

Note this is a **thin idempotency layer** (cache the response, don't re-run) rather than the more robust pattern of a dedicated `idempotency_keys` table with a `PROCESSING/COMPLETED` status and a unique DB constraint. The trade-off: it's simpler, but there's a race window — two concurrent requests with the same key, both checking Redis before either has written back, can both fall through and execute twice. Good enough for a human clicking a button twice; not safe for a high-concurrency payment API.

The state machine itself provides a second, cheaper line of defense here: `approveAction` / `executeAction` both check “is this action already in the target status?” and return early if so — so even without the idempotency key, a duplicate call after the first one *committed* is a safe no-op. The idempotency cache mainly protects the window *during* the first call (e.g., a retry that arrives before the DB commit).

2.5 Cache-Aside (Read-Through) Pattern — Redis in Query Service

`IncidentQueryService.getIncidentById()` :

1. Check Redis: `incident:{id}`
2. Hit → return cached DTO directly
3. Miss → query MySQL → build DTO → SET into Redis with a TTL → return

Two cache tiers exist with different TTLs, tuned to their volatility: - `incident:{id}` — **10s TTL**, single-incident detail. - `incident:active` — **5s TTL**, the list of all non-terminal incidents (the dashboard's most frequently polled endpoint).

Active invalidation, not just TTL expiry. Every Kafka consumer that mutates the read model calls `evictIncidentCache(id)` immediately after `save()` , which deletes both `incident:{id}` and `incident:active` . This means the TTLs are a *safety net* for cache entries nobody explicitly invalidated (e.g., a bug, or an event delivered out of band) — not the primary consistency mechanism. Under normal operation, a write updates MySQL, evicts Redis, and the very next read repopulates the cache with fresh data — so staleness is bounded by “time between Kafka event consumption and cache eviction,” which is milliseconds, not the TTL window.

Redis serialization detail worth knowing cold: `RedisTemplate<String, Object>` is configured with `GenericJacksonJsonRedisSerializer` + `BasicPolymorphicTypeValidator.allowIfSubType(Object.class)` . Generic `Object` -typed Redis templates need **polymorphic type info embedded in the JSON** (an `@class` field) so Jackson knows what Java type to deserialize back into — a plain `ObjectMapper` would just hand you a `LinkedHashMap` . This is also *why* `IncidentReadDto` needed `@Setter` / `@NoArgsConstructor` / `@AllArgsConstructor` added — Jackson's polymorphic deserializer needs a mutable, no-arg-constructible target, which a pure `@Builder` -only immutable DTO doesn't offer.

2.6 Sequence-Number Based Ordering / Idempotent Event Consumption

Every `IncidentEvent` carries a monotonically increasing `sequenceNumber` (an in-process `AtomicLong` in Command Service, incremented once per publish). Every read-model row stores `lastSequenceNumber` . Every consumer method does:

```
if (!isNextInSequence(model.getLastSequenceNumber(), event.getSequenceNumber())) {
    log.warn("Discarding out-of-sequence event...");
    return;
}
```

where `isNextInSequence` just checks `incoming > last` (strictly greater, not necessarily +1 — it tolerates gaps, it just rejects going backwards or replaying the same or an older event).

Why this exists at all, given Kafka already guarantees in-order delivery within a partition: Kafka guarantees order of *production and storage*, not order of *consumption* under all failure modes. A `DefaultErrorHandler` with retries, a consumer rebalance mid-batch, or a manually-triggered DLQ replay (see §2.7) can cause an event to be **redelivered or delivered late**. The sequence check makes the read-model update **idempotent and order-safe** regardless of how many times an event physically arrives or in what order retries land — it's the read-side equivalent of the write-side idempotency key.

2.7 Dead-Letter Queue (DLQ) — Two Layers, Two Different Jobs

This is one of the more sophisticated pieces of the system, and worth understanding as **two separate DLQ mechanisms that happen to interact**:

Layer 1 — Kafka consumer fault tolerance (Query Service). `KafkaErrorHandlingConfig` wires a `DefaultErrorHandler` with `FixedBackOff(1000ms, 3 retries)` around every `@KafkaListener`. If a listener throws (bad payload, DB constraint violation, whatever), Spring Kafka retries 3 times with a 1s pause, and if it still fails, hands the record to `PersistingDeadLetterRecoverer` — a custom subclass of `DeadLetterPublishingRecoverer` that does **two things**, not the framework default's one: 1. Persists the raw failed record + exception message into a MySQL `dlqRecord` table (queryable via `DlqAdminController` / the frontend's DLQ admin page). 2. Also forwards it to Kafka's own convention DLQ topic (`{original-topic}.dlq`).

Persisting to a queryable DB table on top of the standard Kafka DLQ topic is a deliberate choice: a DLQ topic alone is opaque without a consumer built specifically to browse it, whereas a DB row is instantly visible in a REST endpoint / admin UI, and survives independent of Kafka retention settings.

Layer 2 — the remediation action itself. `CLEAR_KAFKA_DLQ` is one of the three real actions the AI agent can propose. When approved and executed, `KafkaAdminService.backupAndClearDeadLetterQueue()`: 1. Consumes **every** record currently sitting in `{service}.dlq` using a throwaway `AdminClient` / `KafkaConsumer` (own consumer group, `UUID`-suffixed so it never collides with a real consumer group and never commits offsets against the real topic). 2. Republishes them verbatim into a new backup topic named `{service}.dlq.rollback.{actionId}` (typo preserved from the source — `rollback`, not `rollback`). 3. Only then deletes the records from the original topic (`AdminClient.deleteRecords`, using `RecordsToDelete.beforeOffset(latestOffset)` per partition — this is a **prefix truncation** to the DLQ's end offset at the moment of clearing, not a topic wipe). 4. Stores `dlq_backup_topic={backupTopicName}` in the action's `previousState` column.

This gives the platform a genuine **compensating-transaction / rollback path**: `performRollbackAction()` reads `previousState`, and if the action was a DLQ clear, calls `restoreDeadLetterQueueFromBackup()` which replays the backup topic's contents straight back into the original `.dlq` topic. The backup topic is the “undo log” for an operation that would otherwise be destructive and unrecoverable.

Two DLQ bugs were found and fixed during development (see project memory): (1) `clearDeadLetterQueue` originally called `describeTopics` before checking `listTopics()`, throwing `UnknownTopicOrPartitionException` on a DLQ that hadn't been created yet; (2) the backup step had an inverted `if (records.isEmpty())` condition, so it only backed up when there was *nothing* to back up — the exact opposite of the intended safety net. Both are good illustrations of how DLQ/rollback code paths are easy to get backwards and hard to catch without deliberately exercising the “topic doesn't exist yet” and “DLQ actually has data” cases in testing.

2.8 Circuit Breaker (Resilience4j) — Agent Service → Prometheus

`MetricsClient.fetchMetrics()` is wrapped with `@CircuitBreaker(name = "metricsService", fallbackMethod = "fallbackToCache")`.

`Config(application.yaml)`:

```
resilience4j:
  circuitbreaker:
    instances:
      metricsService:
        sliding-window-size: 10
        failure-rate-threshold: 50
        wait-duration-in-open-state: 30s
        permitted-number-of-calls-in-half-open-state: 3
        automatic-transition-from-open-to-half-open-enabled: true
```

Mechanics: the breaker tracks the last 10 calls. If $\geq 50\%$ failed, it flips **CLOSED** → **OPEN**. While open, every call to `fetchMetrics` is short-circuited straight to `fallbackToCache()` without even attempting Prometheus — for 30 seconds. After 30s it goes **half-open** and lets 3 calls through as a probe; if they succeed, closes; if not, reopens.

The fallback doesn't return an error or an empty object — it reads the **last known-good MetricSnapshot from Redis** (written by the last

successful call) and returns it with `stale = true` set. `PrometheusAnomalyWatcher.monitorPrometheusMetrics()` explicitly checks `metrics.isStale()` and **skips auto-incident-creation** on stale data — this is the critical bit: a struggling Prometheus doesn't get treated as “everything is now reporting zero errors” (which would silently mask an outage) nor as grounds to fire on last week's numbers. It just backs off and waits for the breaker to recover.

This is a textbook **graceful degradation** pattern: the dependency (Prometheus) being unavailable degrades the system to “can't detect *new* anomalies right now” rather than cascading into false incidents or throwing 500s up the call chain.

2.9 Provider Failover — AI Model Fallback Chain

`aiService.call()` is a simple manual fallback, not a Resilience4j-managed one:

```
try {
    return geminiChatClient.prompt().user(prompt).tools(tools).call().content();
} catch (Exception ex) {
    try {
        return groqClient.prompt().user(prompt).tools(tools).call().content();
    } catch (Exception e2) {
        throw new RuntimeException("Both gemini and groq is failed", e2);
    }
}
```

Gemini (`gemini-2.5-flash`) is primary; Groq (`llama-3.3-70b-versatile`, via an OpenAI-compatible endpoint at `api.groq.com/openai/v1`) is the secondary. Both are wired as **Spring AI `ChatClient` beans** pointed at different model backends, and both are handed the *same* `AgentTools` bean as their toolset — so the fallback is transparent to the tool-calling contract; the second model gets the exact same function-calling surface as the first.

Why manual try/catch instead of Resilience4j `@Retry` / `@CircuitBreaker` here too? This is a fair thing to flag as an inconsistency in an interview: two different resilience mechanisms exist in the same codebase for conceptually similar problems (external dependency might fail → have a fallback). A more consistent design would wrap this in a Resilience4j `CircuitBreaker` + a fallback-provider chain, gaining sliding-window failure tracking and the ability to *stop calling Gemini for a while* after repeated failures instead of re-attempting it on every single request. As written, every call pays Gemini's full timeout on every failure before falling back — there's no “give up on Gemini for 30s” memory between requests.

2.10 Token Bucket Rate Limiting — API Gateway

`RateLimitFilter` (a `OncePerRequestFilter`, so it runs before Spring Security's own filter chain reaches business logic) resolves a **tier** per request:

- `/api/auth/**` → **AUTH** tier — tight limit (10 requests / 60s), because auth endpoints are a brute-force target.
- everything else → **GENERAL** tier — looser (300 requests / 60s).

`Bucket4j` (`Bandwidth.classic` + `Refill.greedy`) implements the actual token bucket per client key. The **client key** is derived as `user:{jwtUsername}` when a valid Bearer token is present (so rate limits follow the authenticated user across IPs/devices), falling back to `ip:{X-Forwarded-For or remoteAddr}` for unauthenticated traffic. Buckets are held in two `ConcurrentHashMap<String, Bucket>` — one per tier — created lazily via `computeIfAbsent`.

Standard rate-limit response headers (`X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset`, and `Retry-After` on a 429) are set on every response, not just rejections — this lets a well-behaved client self-throttle before it ever gets a 429.

Trade-off to know cold: this is **in-memory, per-instance** rate limiting. It is correct for a single API Gateway container (as deployed here). It is **not** correct the moment you run more than one gateway replica behind a load balancer — each instance has its own bucket map, so a client could get up to $N \times \text{limit}$ requests across N instances. The fix in a real multi-instance deployment is a Redis-backed token bucket (`Bucket4j` supports this via a distributed proxy manager) so all instances share bucket state.

2.11 Stateless JWT Authentication

`JwtAuthFilter` runs before Spring Security's `UsernamePasswordAuthenticationFilter`, extracts a Bearer token, validates it, and manually populates `SecurityContextHolder` with a `UsernamePasswordAuthenticationToken` built from `UserDetails` — no session, no server-side token store. `SecurityConfig` sets `SessionCreationPolicy.STATELESS` explicitly. Combined with `RateLimitFilter` (added *before* `JwtAuthFilter` in the chain), the order is: **rate limit** → **authenticate** → **authorize** → **route**, so an attacker can't burn compute on authentication logic before hitting the rate limiter.

Filter chain order, spelled out:

```
Request → RateLimitFilter → JwtAuthFilter → UsernamePasswordAuthenticationFilter → ... → Controller
```

2.12 API Gateway / Routing Pattern

`api-gateway` uses **Spring Cloud Gateway MVC** (`spring.cloud.gateway.server.webmvc` , the Servlet-based variant, not the reactive `WebFlux` `spring-cloud-starter-gateway`). Routes are declarative path-predicate mappings:

```
routes:
  - id: command-service
    uri: http://command-service:8081
    predicates: [ Path=/api/incidents/** ]
  - id: query-service-rest
    uri: http://query-service:8082
    predicates: [ Path=/api/query/** ]
  - id: agent-service
    uri: http://agent-service:8083
    predicates: [ Path=/api/agent/** ]
```

The gateway is the **only** externally-reachable HTTP entry point for authenticated traffic (all `18081-18083` service ports are exposed on the host mainly for local dev/Swagger convenience — a production deployment would keep them internal-network-only and force everything through `18080`). This centralizes JWT validation and rate limiting in one place instead of duplicating that logic into every downstream service — downstream services trust that a request reaching them already passed the gateway’s checks (in this codebase they don’t re-validate JWTs internally, which is a legitimate “the network boundary is the trust boundary” design as long as the internal Docker network truly isn’t reachable from outside).

2.13 Human-in-the-Loop Approval Workflow

This is the safety-critical pattern the whole platform is built around: **the AI can never directly execute anything.**

`AgentTools.proposeAction()` ’s only effect is an HTTP call to `POST /api/incidents/{id}/actions` , which creates a `RemediationAction` in `PROPOSED` status and moves the incident to `ACTION_PROPOSED` . Nothing happens to any container or Kafka topic until a human calls `POST /actions/{id}/approve` (state → `WAITING_APPROVAL` , records `approvedBy`) and then, separately, `POST /actions/{id}/execute` (which is where `performRealAction()` actually touches Docker/Kafka).

Three distinct API calls — propose, approve, execute — each gated by the state machine, each independently idempotent — instead of one “do it” endpoint. This is deliberate defense in depth: even a compromised or hallucinating agent can only ever create a *suggestion* sitting in a queue for a human to review.

2.14 Compensating Action / Rollback Pattern

Rather than true distributed transactions (impractical across Docker + Kafka + MySQL), the system uses the **Saga-style compensating action** pattern: every executed action records a `previousState` snapshot at execution time (`replicas=2` , or `dlq_backup_topic=...`), and rollback is a *second, independent forward operation* that uses that snapshot to reverse the effect — not a “transaction abort.” `rollbackAction()` also creates a **new synthetic action row** (`ROLLBACK_{originalType}` , linked via `rollbackOf`) rather than mutating the original action’s type — preserving a complete, honest audit trail: you can always see both what was originally done and what was done to undo it, as two separate timeline entries.

Per action type, the compensation differs: - `RESTART_SERVICE` → **no real inverse** (a restart isn’t reversible — “un-restarting” is meaningless), so rollback is a documented no-op that still records `EXECUTED` for audit purposes. - `SCALE_WORKER_PODS` → scale back to the recorded `replicas=N` snapshot. - `CLEAR_KAFKA_DLQ` → replay the backup topic (see §2.7).

2.15 Real-Time Push via WebSocket (STOMP), Not Polling

`WebSocketEventRelay` uses `SimpMessagingTemplate` to broadcast to two STOMP destinations: `/topic/incidents/{id}` (single-incident updates) and `/topic/incidents/active` (the dashboard’s live list). Every Kafka consumer method in Query Service, right after persisting the read-model change and evicting the cache, pushes the same payload over both channels. The frontend’s `useWebSocket` hook subscribes once and receives push updates — no dashboard polling loop, no stale-until-next-refresh UX. This closes the CQRS eventual-consistency gap (§2.1) from the user’s perspective: the round trip from “Command Service commits” to “browser re-renders” is one Kafka hop plus one WebSocket push, typically tens of milliseconds.

2.16 AI Tool-Calling (Function Calling) Pattern

`AgentTools` exposes six `@Tool`-annotated methods to the model via Spring AI: five **read-only diagnostics** (`getServiceHealth`, `getKafkaConsumerLag`, `getRedisMemoryStats`, `checkDatabaseDeadlocks`, `getRecentDeployments`) and exactly one **write** capability (`proposeAction`, and even that only *proposes*, per §2.13). The system prompt explicitly instructs the model to “*Investigate using the available read-only diagnostic tools before deciding on anything... Call as many diagnostic tools as you need to form a grounded hypothesis*” — this is a **ReAct-style loop** (Reason → Act → Observe, repeated) implemented for free by Spring AI’s `ChatClient.tools()`, which handles the multi-turn tool-call/tool-result exchange with the model internally until it returns a final text answer.

`AgentTools` is a Spring singleton bean with a mutable `currentIncidentId` field (`@Setter`), set by `IncidentEventListener` right before each `aiService.call()`. **This is a concurrency trap worth naming explicitly:** if two `incident.created` events are processed concurrently (e.g., the listener’s container has `concurrency > 1`, or Kafka rebalances two partitions to the same consumer), one incident’s tool calls could pick up another incident’s ID, because the field is shared mutable state on a singleton, not scoped per-request. It works correctly today because the consumer is effectively single-threaded per partition and this is a low-volume path, but it is not safe to raise consumer concurrency without first making incident context request-scoped (e.g., pass `incidentId` as a tool-call parameter set by the LLM itself, or use a `ThreadLocal` /per-invocation context object instead of a shared field).

Currently the five diagnostic tools return **randomized mock data** (`new Random().nextInt(...)`) rather than live queries — a clearly-labeled placeholder for wiring to real Prometheus/Kafka-admin/MySQL queries, which is honest to call out rather than obscure in an interview: the *tool-calling plumbing and human-approval safety envelope* are fully real and production-shaped; the *diagnostic data source* is a stub.

3. Distributed Tracing — How It Actually Works

3.1 The Pipeline

```
Service (Spring Boot + Micrometer Tracing)
  → OTLP/HTTP export (traces, logs, metrics)
  → otel-collector (receives OTLP, batches, tags with deployment.environment)
    ├── traces → Tempo (OTLP/gRPC :4317)
    ├── logs   → Loki (OTLP/HTTP, /otlp endpoint)
    └── metrics → Prometheus (remote_write)
  → Grafana (datasources: Tempo + Loki + Prometheus, TraceQL editor enabled)
```

Every one of the 5 Spring Boot services carries the same `management.tracing.otlp/opentelemetry` config block pointed at `http://otel-collector:4318` (except gRPC to 4317 for the collector→Tempo hop) — see `agent-service/application.yaml $Observability`. `management.tracing.sampling.probability: 1.0` means **100% of requests are traced** — correct for a dev/demo environment where trace volume is low and completeness matters more than collector load; a production system at real traffic would sample (e.g., 1-10%, often with “always trace errors” overrides) to control cost and collector throughput.

3.2 Context Propagation Across Process Boundaries

Two different transports carry the trace context across services, and each does it differently:

- **HTTP hops** (Frontend → Gateway → Command/Query/Agent Service, Agent Service → Command Service via `RestClient`): Micrometer Tracing auto-instruments `RestClient` / `RestTemplate` / `Servlet` filters to inject/extract the W3C `traceparent` header automatically. No code in this repo manually touches trace headers — it’s transparent AOP-style instrumentation from Spring Boot’s Actuator + Micrometer starter.
- **Kafka hops** (Command Service → Query/Agent Service via `incident.created` etc.): `spring.kafka.listener.observation-enabled: true` turns on Micrometer’s Kafka observation instrumentation, which propagates trace context through Kafka record **headers** (not the message body) on both produce and consume. This is what lets a single trace span the entire chain: *HTTP request creates incident → Kafka publish → Kafka consume in Agent Service → AI tool calls → HTTP call back to Command Service to propose an action* — all one trace in Tempo, even though it crosses an async message broker in the middle.

3.3 Log ↔ Trace Correlation

Every service’s logging pattern is:


```
logging:
  pattern:
    level: "%5p [%${spring.application.name},%X{traceId:-},%X{spanId:-}]"
```

Micrometer Tracing populates the SLF4J **MDC** (`%X{...}`) with the current `traceId` / `spanId` for the duration of a request/message-processing scope, and the log pattern prints them on every line. Combined with `OpenTelemetryLogbackConfig` shipping logs to Loki via OTLP, this means: given a slow or failed trace in Tempo, you can copy its `traceId` and pivot straight to Loki to see every log line — across all 5 services — that was emitted while handling that exact request. This trace ↔ log correlation is the single most valuable debugging capability the observability stack buys, and it’s essentially free once the MDC pattern and OTLP log export are wired — no manual `logger.info("traceId=" + ...)` anywhere.

3.4 Metrics ↔ Trace Correlation (Exemplars)

`aic-prometheus` is started with `--enable-feature=exemplar-storage`, and Tempo’s `metrics_generator` is configured to derive RED-style metrics (`service-graphs`, `span-metrics`) from the trace stream itself and remote-write them into the same Prometheus instance **with exemplars** (`send_exemplars: true`). An exemplar is a concrete trace ID attached to a specific point on a metrics graph — so in Grafana, hovering a latency spike on a Prometheus-backed panel can jump straight to the *actual trace* that produced that data point. This is the third leg of the “three pillars of observability” triangle (metrics, logs, traces) being genuinely cross-linked rather than three disconnected dashboards.

3.5 Service Graphs

Tempo’s `metrics_generator` with the `service-graphs` processor derives a live **service dependency graph** purely from observed trace edges (who called whom, how often, at what latency/error rate) — no manually-maintained architecture diagram required; it’s derived from what’s actually happening in production traffic.

4. Resilience — The Full Picture

Resilience here isn’t one mechanism — it’s several, each aimed at a different failure mode:

Failure mode	Mechanism	Where
Duplicate client request (double-click, retry-after-timeout)	Idempotency key cache (Redis, 24h TTL)	Command Service, <code>approve / execute</code>
Transient Kafka consumer failure (bad payload, momentary DB blip)	<code>DefaultErrorHandler + FixedBackOff(1s, 3 tries)</code>	Query Service listeners
Permanent consumer failure after retries exhausted	DLQ: persist to MySQL + forward to <code>{topic}.dlq</code>	Query Service
Out-of-order / redelivered event	Sequence-number check, monotonic-only apply	Query Service consumers
Downstream dependency (Prometheus) flapping or down	Circuit breaker (Resilience4j) + stale-cache fallback	Agent Service <code>MetricsClient</code>
Primary AI provider outage/error	Manual try/catch fallback to secondary provider	Agent Service <code>AIService</code>
Docker/Kafka execution failure mid-remediation	Soft-fail: log + return <code>true</code> , action still recorded <code>EXECUTED</code>	Command Service <code>performRealAction</code>
A bad remediation was executed	Rollback / compensating action using <code>previousState</code> snapshot	Command Service <code>performRollbackAction</code>
Client hammering an endpoint	Token-bucket rate limiting, tiered by sensitivity	API Gateway
Cache staleness after a write	Active cache eviction on every mutation (not just TTL)	Query Service
Container crash / restart	<code>restart: unless-stopped</code> on every compose service	<code>docker-compose.yml</code>

The soft-fail pattern deserves its own callout because it’s counter-intuitive. Look closely at `performRealAction`:

```

boolean success = dockerExecutionService.restartService(serviceName);
if (!success) {
    log.warn("... Action recorded successfully.");
    return true; // <-- returns true even though the Docker call failed!
}
return success;

```

This is a **deliberate demo/dev-environment concession**, not an accident: the target container named after `serviceName` frequently doesn't exist in the local Docker daemon (e.g., the AI proposes `RESTART_SERVICE` for a service that isn't actually running as a container in this compose stack), and the design choice was “don't let a missing/mismatched target container block the incident lifecycle from progressing through its states for demonstration purposes.” **This is a real trade-off to be upfront about in an interview:** in a genuine production system, silently treating “the remediation didn't happen” the same as “the remediation succeeded” is dangerous — it would tell an operator an incident was remediated when it wasn't. The honest fix is to introduce a distinct `EXECUTION_FAILED` / `NO_TARGET_FOUND` outcome instead of collapsing “not found” into “succeeded,” and surfacing it distinctly in the UI and in escalation logic.

5. How Docker Execution Actually Works

This is the part most people gloss over, so it's worth being precise.

5.1 Two Completely Different Docker Integration Paths

The system talks to Docker **two different ways**, for two different jobs, and this is a design decision worth explaining out loud:

(A) `docker-java` client library — for single-container operations (restart, inspect, list).

`DockerConfig` builds a `DockerClient` bean using `docker-java`'s low-level HTTP client over the Unix domain socket (unix:///var/run/docker.sock, mounted read-write into the `command-service` container via `docker-compose.yml`'s `volumes` section). This gives typed Java method calls (`dockerClient.restartContainerCmd(id).exec()`, `.listContainersCmd().exec()`, `.inspectContainerCmd(id).exec()`) directly against the Docker Engine API — fast, no subprocess spawning, structured responses.

A real bug was hit and fixed here: the default transport (`docker-java`'s Apache HttpClient5-based transport) failed with **connection refused talking to the Unix socket** from inside the container. The fix was switching the transport implementation to `docker-java-transport-zerodep` (`ZerodepDockerHttpClient`), a dependency-free transport that handles Unix domain sockets correctly in this containerized setup. This class of bug — an HTTP client library that works fine talking TCP but mishandles Unix sockets — is common enough with Docker-in-Docker/socket-mounted setups to be worth remembering by name.

(B) Raw `docker compose` CLI via `ProcessBuilder` — for compose-level scaling.

`scaleWorkerPods()` does **not** use `docker-java` at all. It shells out:

```

ProcessBuilder pb = new ProcessBuilder(
    "docker", "compose", "-p", composeProject,
    "up", "-d", "--scale", composeService + "=" + replicas, "--no-recreate"
);
pb.directory(new File(containerPath));

```

Why not do this through the Docker Engine API too? Because “scale a Compose service to N replicas” isn't a primitive the Docker Engine API exposes directly — it's a *Compose-level* concept (Compose decides container naming, networking, and reconciliation for you). `docker-java` talks to the Engine API, one level below Compose; replicating Compose's scale semantics by hand via raw container create/start calls would mean reimplementing a chunk of Compose's own reconciliation logic. Shelling out to the real `docker compose` binary — the same tool an operator would run by hand — is the pragmatic choice: it guarantees identical behavior to a human running `docker compose up -d --scale ...=3`, at the cost of losing typed responses (you get raw stdout/exit-code instead) and paying subprocess-spawn overhead.

`--no-recreate` matters: it tells Compose not to touch already-running containers matching the current config, only add/remove replicas to hit the target count — so scaling doesn't cause an unrelated restart-with-downtime of the existing replicas.

5.2 The Cross-Project Scaling Trick

The interesting engineering problem: `command-service` runs inside *its own* Docker Compose project (this repo's `docker-compose.yml`), but the service it needs to scale (e.g., a worker service from the separate [[distributed-job-forge]] project) lives in **a different Compose project entirely**, with its own compose file on a different part of the host filesystem. `scaleWorkerPods()` solves this by:

1. `findContainerId(serviceName)` — substring-match the target container’s name against all containers visible to the Docker *daemon* (containers are daemon-global, not project-scoped — this is what makes step 2 possible at all).
2. `dockerClient.inspectContainerCmd(id)` → read the container’s labels. Docker Compose automatically stamps every container it creates with `com.docker.compose.project`, `com.docker.compose.service`, and `com.docker.compose.project.working_dir` labels. This is the load-bearing trick: **the target container itself tells you which Compose project and working directory it belongs to** — no hardcoded config mapping every service name to its project needed.
3. The `working_dir` label is a **host filesystem path** (e.g., `/home/user/distributed-job-forge`), but `command-service` is running *inside a container* and can’t `cd` to a host path directly. `resolveContainerPath()` translates it via `scaling.path-mappings` — a config map (`ScalingProperties`, bound from YAML) from host path → the path where that other project’s directory is bind-mounted into *this* container (`docker-compose.yml: ${DJF_PROJECT_DIR}:/djf-workspace:ro`).
4. Only now can `ProcessBuilder` run `docker compose -p {project} up -d --scale ...` with `pb.directory(/djf-workspace)` — a path that exists inside *this* container and, thanks to the read-only bind mount, contains the target project’s actual `docker-compose.yml`.

Why YAML [key] bracket syntax for the path-mapping config, called out specifically in project notes: a normal YAML map key like `/home/user/djf: /djf-workspace` is ambiguous to a YAML parser because the key contains `:` and `/` — bracket syntax (`[/home/user/djf]: /djf-workspace`) forces it to be parsed as a literal flow-scalar key rather than accidentally splitting on an embedded colon.

What this buys, architecturally: `command-service` can execute real infrastructure operations against a completely independent project’s containers without that project needing any awareness of AI Incident Commander, any shared network, or any API — it only needs (a) to be a Docker Compose project on the same host/daemon, and (b) its directory bind-mounted read-only for the CLI to find its compose file. The trade-off is fragility: it depends on Compose’s labeling convention staying stable, on the bind-mount path mapping being kept in sync by hand whenever `DJF_PROJECT_DIR` changes, and on both projects sharing one Docker daemon (this doesn’t work in a real multi-host/Swarm/Kubernetes deployment — there, this whole mechanism would be replaced by calling the Kubernetes API to scale a Deployment, which has no analogous “which host filesystem is this on” problem).

5.3 Snapshot-Before-Mutate, for Every Reversible Action

Before scaling, `getCurrentReplicationCount()` counts currently-running containers matching the service name and stores `replicas={N}` on the action row *before* the scale call runs. This is the same “capture state before you touch it” discipline as the DLQ backup-before-clear (§2.7) — both exist purely to make rollback (§2.14) possible later. It’s a pattern worth generalizing: **any action that claims to be reversible needs its pre-image captured at execution time, not reconstructed after the fact** (reconstructing “what was the replica count before we scaled” *after* scaling is not generally possible).

6. End-to-End Walkthroughs

6.1 Happy path: alert → AI proposal → human approval → real execution

1. Prometheus fires (either pushed via `POST /api/incidents/webhook/prometheus`, or detected by `PrometheusAnomalyWatcher`’s 15s poll loop when error-rate/latency crosses threshold and the 3-minute per-service cooldown has elapsed).
2. `createIncident()` persists `Incident(status=NEW)`, publishes `incident.created` keyed by the new incident ID.
3. **Two consumers** react to that one event independently: Query Service projects it into the read model (dashboard now shows it, pushed live via WebSocket); Agent Service’s `IncidentEventListener` sets `AgentTools.currentIncidentId` and calls `AiService.call()` with an investigation prompt.
4. The model calls 0-N diagnostic tools (`getServiceHealth`, `getKafkaConsumerLag`, etc.), reasons over the results, then calls `proposeAction(actionType, rationale)` — which is a plain REST `POST` back to Command Service.
5. `proposeResponse()` transitions `NEW/INVESTIGATING` → `ACTION_PROPOSED`, creates a `RemediationAction(status=PROPOSED)`, publishes `action.proposed`.
6. A human sees the proposal on the dashboard (pushed via WebSocket through the same fan-out) and calls `POST /actions/{id}/approve` with an idempotency key. State → `WAITING_APPROVAL`; `action.approved` published.
7. Human calls `POST /actions/{id}/execute`. `performRealAction()` dispatches on `actionType` string content (`RESTART / SCALE | POD | WORKER / DLQ | LETTER | CLEAR | PURGE`) to the matching Docker/Kafka operation, snapshotting `previousState` first. State → `EXECUTING`; `action.executed` published.
8. Every one of these publishes is independently consumed by Query Service (read model + cache evict + WebSocket) and would be consumed by Notification Service (email) — the dashboard and any subscribed human stay in lockstep with the write side throughout, without polling.

6.2 Something went wrong: rollback

Operator (or a future automated monitor watching post-execution metrics — the `MONITORING` → `ROLLBACK` transition exists in the state machine for exactly this) calls `POST /actions/{id}/rollback`. `performRollbackAction()` reads `previousState`, reverses the specific side effect (scale back down, or replay the DLQ backup topic), records a **new** `ROLLBACK_{type}` action row, and publishes three events in sequence (`action.rolled_back`, then a synthetic `action.proposed` + `action.executed` for the rollback action itself) — because from the read model’s perspective, “a rollback happened” is best represented as a normal new action entry on the timeline, not a mutation erasing the original one.

7. Design Trade-offs — Summary Table

Decision	Benefit	Cost / Risk
CQRS across two separate services+DBs	Independent scaling, fault isolation, read model shaped for UI	Eventual consistency; two schemas to keep mentally in sync; more moving parts to deploy
Kafka fan-out for all consumers	Zero-coupling between publisher and consumers; easy to add new consumers	No exactly-once guarantee across the whole pipeline; ordering only guaranteed per-partition-key
In-memory <code>AtomicLong</code> sequence counter	Simple, zero infra	Resets to 0 on Command Service restart — a restart mid-flight could make sequence numbers look “out of order” to consumers that already saw higher numbers pre-restart, silently dropping legitimate post-restart events until they overtake the old high-water mark. A persisted or DB-generated sequence (or just Kafka’s own offset) would survive restarts.
Idempotency cache in Redis, no dedicated table	Simple, fast, TTL self-cleans	Not airtight under true concurrent duplicate requests (race on first write); relies on Redis availability
Redis cache-aside with short TTL + active eviction	Fast reads, bounded staleness even if eviction is missed	Two invalidation paths (TTL + explicit) to reason about; Redis outage degrades to “always miss,” not “wrong answer” — acceptable, but Redis becomes a latency dependency on every read
Circuit breaker only around Prometheus, plain try/catch around AI providers	Correct behavior in both cases	Inconsistent resilience tooling in one codebase; the AI fallback has no “stop hammering a known-down provider” memory
Soft-fail Docker/Kafka execution (<code>return true</code> on failure)	Demo/dev incident lifecycle never gets stuck on a missing target container	Genuinely dangerous in production — an incident can be marked EXECUTED when nothing actually happened. Needs a distinct failure status before this ships for real.
Shelling out to <code>docker compose</code> CLI for scaling	Identical behavior to a human operator; no Compose-semantics reimplementaion	Subprocess overhead, untyped output parsing, depends on the <code>docker</code> CLI being present in the container image and the socket being mounted
Cross-project scaling via container labels + path mapping	Zero coupling to the target project’s code; daemon-level container labels are always accurate	Fragile to path-mapping config drift; only works single-host with one shared Docker daemon — doesn’t generalize to Kubernetes/Swarm
Rate limiting in-memory per gateway instance	Zero extra infra, low latency	Breaks down (limits multiply) the moment the gateway is scaled beyond 1 replica; needs a distributed backing store (Redis) to scale out correctly
100% trace sampling	Complete visibility, nothing missed in dev	Would not scale cost/throughput-wise at real production traffic; needs a sampling strategy before scaling up
Human-in-the-loop mandatory approval	Hard safety boundary — AI cannot directly mutate infrastructure	Slower MTTR than full autonomy; deliberate trade-off given the blast radius of infra actions
<code>AgentTools</code> singleton with mutable <code>currentIncidentId</code>	Simple to wire, works under today’s low/sequential load	Not safe under concurrent multi-incident processing; needs per-invocation context if consumer concurrency is ever raised

8. Interview Question Bank

Organized by topic. Each is the kind of question this project should let you answer *with a specific example from the code*, not just a definition.

CQRS & Event-Driven Architecture

1. Why did you split Command and Query into separate services instead of separate packages in one Spring Boot app? What would you lose by merging them back?
2. Walk me through what happens, end to end, between a `POST /approve` returning `200` and the dashboard visually updating. Where exactly is the consistency gap, and how did you close it?
3. What guarantees does Kafka actually give you about event ordering, and where in your code do you depend on that guarantee? What's your partitioning key and why?
4. If you added a fourth consumer tomorrow (say, a Slack notifier), what would you have to change in Command Service? What does that tell you about coupling in this design?
5. Your read model is a full second MySQL database, rebuilt from Kafka events. If you lost that database entirely, could you rebuild it? What would you need to guarantee that (hint: Kafka retention/compaction)?

Idempotency & Consistency

6. Explain the difference between your idempotency-key mechanism and your sequence-number mechanism. They sound similar — why do you need both?
7. Where's the race condition in your current idempotency implementation? How would you close it with a stronger primitive?
8. Why do you check `if (incoming > last)` rather than `if (incoming == last + 1)` in your consumers? What failure mode does the strict-equality version break on that the greater-than version tolerates?
9. Your sequence counter is an in-memory `AtomicLong`. What happens to correctness on a Command Service restart? How would you fix it?

Resilience Patterns

10. Explain your circuit breaker configuration line by line — sliding window, failure threshold, wait duration, half-open probes. Walk me through a full open → half-open → closed cycle.
11. Why does the Prometheus circuit breaker's fallback return stale cached data with a flag, rather than throwing or returning nulls/zeros? What would go wrong with each alternative?
12. Your AI provider fallback (Gemini → Groq) is a manual try/catch, but your Prometheus client uses Resilience4j. Why the inconsistency, and what would you change?
13. Defend, or attack, the `return true` soft-fail behavior in `performRealAction` when a target container isn't found. What's the argument for it? What's the argument against it in production?
14. What's the difference between a retry, a circuit breaker, and a dead-letter queue? Where does each show up in this system, and why is one alone not sufficient?

Distributed Tracing & Observability

15. Trace context needs to survive an HTTP hop and a Kafka hop in this system. Explain how propagation differs between the two transports.
16. What's in a trace's MDC that lets you pivot from Tempo to Loki? What would break if you removed the logging pattern's `%X{traceId}`?
17. What's an exemplar, and where does one come from in this stack? Why does it require Tempo's metrics-generator rather than just Prometheus scraping `/metrics`?
18. Why is 100% trace sampling fine here but wrong at scale? What sampling strategy would you introduce, and what would you be careful not to drop?
19. How is the service dependency graph in Grafana generated? What would it show if `agent-service` started silently failing to call `command-service`?

Docker Execution

20. Why does this system use `docker -java` for restarts but shell out to the `docker compose` CLI for scaling, instead of one consistent approach?
21. Explain the container-label trick that lets `command-service` scale containers belonging to a completely separate Compose project it has no other relationship with.
22. Why can't `command-service`, running inside a container, just `cd` to the target project's host directory to run `docker compose scale`? How is that solved?

23. What broke with the default `docker - java` transport talking to the Unix socket, and what fixed it? What class of bug is that, generally?
24. This mechanism only works because everything shares one Docker daemon on one host. What would have to change to make this work across a multi-node Kubernetes cluster instead?

Rollback / Compensating Transactions

25. This system doesn't use real distributed transactions across Docker/Kafka/MySQL. What pattern does it use instead, and why is that the right call here?
26. Why does a rollback create a brand-new action row instead of just flipping the original action's status back?
27. Walk through exactly what's captured in `previousState` for a scale action versus a DLQ-clear action, and how each is used to reverse the operation.
28. Why is `RESTART_SERVICE` rollback a documented no-op? Is that a limitation or a correct modeling of reality?

Security

29. Walk through your filter chain order — rate limiter, JWT filter, Spring Security's own filter — and explain why that specific order matters.
30. Why is your API stateless (`SessionCreationPolicy.STATELESS`)? What would change if you needed to support token revocation?
31. Your rate limiter keys authenticated users by username and everyone else by IP. What's the security reasoning, and what's a way to defeat the IP-based tier?
32. What breaks about your current rate limiter the moment you run two API Gateway replicas behind a load balancer? How would you fix it?

AI / Agent Design

33. Why can the agent only *propose* actions, never execute them? What's the actual blast radius being protected against?
34. Explain the concurrency bug risk in `AgentTools.currentIncidentId`. Under what conditions would it actually manifest, and why doesn't it today?
35. Your diagnostic tools currently return random mock data. If you wired them to real Prometheus/Kafka-admin queries, what would you need to be careful about regarding tool latency and the model's reasoning loop?
36. What is a "ReAct-style" tool-calling loop, and where does Spring AI's `ChatClient.tools()` implement it for you versus what you had to build yourself?

General System Design

37. If incident volume increased 100x, which component breaks first in this architecture, and what's your first scaling move?
38. Where would you add a Kafka schema registry to this system, and what problem would it actually solve that you don't have today (single-team, evolving Java DTOs)?
39. This system currently trusts that only the API Gateway is externally reachable. How would you defend that trust boundary if you had to actually enforce it at the network layer (not just by convention)?
40. If you had to pick one component to rebuild with stronger guarantees before this went to production, which would it be and why?

9. Quick-Reference: "What pattern is this?" Cheat Sheet

If you see...	It's...
Two services, two databases, one Kafka bridge	CQRS
<code>sequenceNumber</code> + <code>isNextInSequence</code>	Idempotent/ordered event consumption
<code>X-Idempotency-Key</code> + Redis	Idempotency key pattern
Redis <code>GET</code> → miss → DB query → <code>SET</code>	Cache-aside
<code>@CircuitBreaker</code> + <code>fallbackMethod</code>	Circuit breaker
try Gemini / catch → try Groq	Failover / fallback chain
<code>FixedBackOff</code> + <code>DeadLetterPublishingRecoverer</code>	Retry + DLQ
backup topic before delete, <code>previousState</code> snapshot	Compensating transaction / Saga-style rollback
Bucket4j <code>Bandwidth</code> / <code>Refill</code>	Token bucket rate limiting
<code>SessionCreationPolicy.STATELESS</code> + Bearer JWT	Stateless authentication
Spring Cloud Gateway <code>routes</code>	API Gateway / reverse proxy pattern
propose → approve → execute, three separate endpoints	Human-in-the-loop / two-person-rule-style gating
<code>@Tool</code> methods + <code>ChatClient.tools()</code>	Function calling / tool use
<code>SimpMessagingTemplate.convertAndSend</code>	Server push (WebSocket/STOMP)
<code>traceparent</code> header, <code>%X{traceId}</code> in logs	Distributed tracing + log correlation
OTLP → Collector → Tempo/Loki/Prometheus	Three-pillars observability pipeline
Docker Compose labels read at runtime	Convention-based service discovery (single-host)